

WSL/Docker GUI表示トラブル - 会話ログ

作成日: 2026-05-29 / 形式: 会話内容を原文ベースでPDF化

User

wsl上のシミュレータを動かして、それをローカルにある表示画面で表示させるのだが、なぜか表示されない。

Assistant

まず切り分けるべきは「GUI転送の仕組みが生きていない」のか、「シミュレータ側が描画バックエンドを間違えている」のかです。WSL2/Windows 11 の WSLg 前提で整理します。

原因はだいたい **4系統** に分かれます。

1. まず最小テスト

WSL 内でこれを実行してください。

```
```bash id="n4y7l7"
echo "DISPLAY=$DISPLAY"
echo "WAYLAND_DISPLAY=$WAYLAND_DISPLAY"
echo "XDG_RUNTIME_DIR=$XDG_RUNTIME_DIR"
ls -l /mnt/wslg
```
```

Windows 11 + WSLg が正常なら、だいたいこういう値が出ます。

```
```bash id="c4nj41"
DISPLAY=:0
WAYLAND_DISPLAY=wayland-0
XDG_RUNTIME_DIR=/mnt/wslg/runtime-dir
```
```

Microsoft 公式では、WSL2 は Linux GUI アプリを Windows 上で統合的に動かせる、つまり X11/Wayland GUI アプリを WSLg 経由で表示できる仕組みになっています。([learn.microsoft.com](https://learn.microsoft.com/en-us/windows/wsl/tutorials/gui-apps?utm_source=chatgpt.com))

次に GUI の最小確認。

```
```bash id="eet39i"
sudo apt update
sudo apt install -y x11-apps mesa-utils
xeyes
```
```

判定

| 結果 | 意味 |
|---|---|
| `xeyes` が表示されない | WSLg / DISPLAY / Wayland / Windows 側の問題 |
| `xeyes` は表示されるがシミュレータだけ出ない | シミュレータ側の描画バックエンド問題 |
| `sudo` で起動した時だけ出ない | 環境変数が root に渡っていない |
| Docker 内から起動した時だけ出ない | WSLg のソケットをコンテナに渡していない |

2. 一番ありそうな原因

君のケースは「WSL 上のシミュレータを動かして、ローカルの表示画面に出す」なので、かなりありそうなのはこれです。

原因A：Wayland と X11 の食い違い

最近の WSLg は Wayland / X11 両方に対応していますが、シミュレータや GUI ライブラリが Wayland にうまく対応していないことがあります。Microsoft 公式にも WSL GUI は X11 と Wayland の両方を扱う仕組みとして説明されています。([learn.microsoft.com](https://learn.microsoft.com/en-us/windows/wsl/tutorials/gui-apps?utm_source=chatgpt.com))

シミュレータ起動前に、いったん X11 側に寄せてください。

```
```bash id="hkehnc"
unset WAYLAND_DISPLAY
export SDL_VIDEODRIVER=x11
export QT_QPA_PLATFORM=xcb
export GDK_BACKEND=x11
```
```

その後、シミュレータを起動。

```
```bash id="i3nh4c"
python your_simulator.py
```
```

特に **pygame / SDL / MuJoCo / GLFW / Qt** 系だと、これで直ることがあります。

3. sudo 経由で起動している場合

例えばこうしている場合。

```
```bash id="e2n5w7"
sudo python simulator.py
```
```

`DISPLAY` や `WAYLAND\_DISPLAY` が root に引き継がれず、GUI が出ないことがあります。

まずは `sudo` なしで起動。

```
```bash id="1rcygl"
python simulator.py
```

```

どうしても sudo が必要なら、一時的にこう試す。

```
```bash id="m2m2yn"
sudo -E python simulator.py
```
```

または root 側で確認。

```
```bash id="yvcgfg"
sudo env | grep -E 'DISPLAY|WAYLAND|XDG'
```
```

何も出ないなら、それが原因です。

4. Docker / コンテナ内で動かしている場合

WSL 上でさらに Docker コンテナ内からシミュレータを動かしているなら、WSLg の表示ソケットをコンテナに渡す必要があります。

例：

```
```bash id="q3h458"
docker run -it --rm -e DISPLAY=$DISPLAY -e WAYLAND_DISPLAY=$WAYLAND_DISPLAY -e
XDG_RUNTIME_DIR=$XDG_RUNTIME_DIR -v /tmp/.X11-unix:/tmp/.X11-unix -v /mnt/wslg:/mnt/wslg
your_image
```
```

Docker 内で GUI が出ない場合、WSL 自体ではなく **コンテナが WSLg のソケットを見えていない** 可能性が高いです。

5. OpenGL / GPU 周りが原因の場合

シミュレータが OpenGL を使う場合、描画コンテキスト生成で落ちていることがあります。

確認：

```
```bash id="f6dszv"
glxinfo -B
```
```

うまくいかない場合は、ソフトウェアレンダリングで試す。

```
```bash id="k7zrv5"
export LIBGL_ALWAYS_SOFTWARE=1
python your_simulator.py
```
```

これで表示されるなら、原因は GUI というより **OpenGL / GPU / Mesa / ドライバ周り** です。

6. WSLg 自体を再起動・更新

PowerShell 側で：

```
```powershell id="0fxodh"
wsl --shutdown
wsl --update
wsl --version
```
```

その後、Ubuntu を再起動して再確認。

```
```bash id="akw8za"
echo $DISPLAY
xeyes
```
```

7. Windows 10 の場合は別問題

Windows 10 だと、WSLg が標準で使える Windows 11 前提とは違い、VcXsrv や Xming などの X server を Windows 側で立てる必要があるケースがあります。

その場合は WSL 側で：

```
```bash id="486mfh"
export DISPLAY=$(ip route | awk '/default/ {print $3}'):0.0
```
```

Windows 側では VcXsrv を起動し、ファイアウォールでブロックされていないか確認します。

ただし **Windows 11 + WSLg なら、基本的にこの手動 X server 設定は不要** です。逆に古い設定が `~/.bashrc` に残っていると壊れることがあります。

確認：

```
```bash id="e6tjbo"
grep DISPLAY ~/.bashrc ~/.profile ~/.zshrc 2>/dev/null
```
```

もしこういうのが残っていたら、一旦コメントアウト。

```
```bash id="nwh43h"
export DISPLAY=...
```
```

まず試す順番

```
```bash id="ubuekq"
1. WSLg環境変数確認
echo $DISPLAY
echo $WAYLAND_DISPLAY
echo $XDG_RUNTIME_DIR

2. 最小GUI確認
sudo apt install -y x11-apps mesa-utils
xeyes

3. シミュレータをX11寄せて起動
unset WAYLAND_DISPLAY
export SDL_VIDEODRIVER=x11
export QT_QPA_PLATFORM=xcb
export GDK_BACKEND=x11
python your_simulator.py

4. OpenGLが怪しければ
export LIBGL_ALWAYS_SOFTWARE=1
python your_simulator.py
```
```

かなり雑に言うと

- `xeyes` も出ない
→ **WSLg / DISPLAY 設定が壊れている**
- `xeyes` は出る
→ **シミュレータ側の描画ライブラリ、Wayland/X11、OpenGL の問題**
- `sudo` や Docker でだけ出ない
→ **環境変数・ソケットが別ユーザー/別コンテナに渡っていない**

まずは `xeyes` が表示されるかで切るのが一番早いです。

User

なんか、あるコンテナでは画面が映るんだけど、別のコンテナでは映らないんだよね。ポートやipアドレスは同じもの使ってるけど。dockerイメージとか、コンテナ作る時の設定も少し違うかもしれない。

Assistant

ここは「ポート/IPが同じ」より、**コンテナ内に GUI 用ソケット・環境変数・描画ライブラリが揃っているか**の差を見るべきです。動くコンテナと動かないコンテナを `docker inspect` とコンテナ内 `env` で比較するのが早いです。

それなら、かなり原因は絞れます。

ポートや IP アドレスが同じでも、GUI 表示には不十分です。

WSLg 経由の GUI は、多くの場合 **TCP/IP のポート通信ではなく、Unix socket / Wayland socket / X11 socket /

OpenGL ライブラリ / 環境変数**に依存します。

WSLg は X11 と Wayland の Linux GUI アプリを Windows 上に表示する仕組みです。つまり、コンテナから表示したい場合も、その WSLg 側のソケットや環境変数をコンテナに渡す必要があります。([github.com](https://github.com/microsoft/wslg?utm_source=chatgpt.com)) Docker 側では、ホスト上のファイルやディレクトリをコンテナに見せるには bind mount が必要です。([docs.docker.com](https://docs.docker.com/engine/storage/bind-mounts/?utm_source=chatgpt.com))

まず見るべき差分

動くコンテナを `good`、動かないコンテナを `bad` とします。

```
```bash id="w9n1a5"
docker inspect good > good.inspect.json
docker inspect bad > bad.inspect.json

docker exec good env | sort > good.env.txt
docker exec bad env | sort > bad.env.txt

diff -u good.env.txt bad.env.txt | less
```
```

特に見るべき環境変数はこれです。

```
```bash id="eaqpun"
DISPLAY
WAYLAND_DISPLAY
XDG_RUNTIME_DIR
PULSE_SERVER
LD_LIBRARY_PATH
QT_QPA_PLATFORM
SDL_VIDEODRIVER
GDK_BACKEND
MUJOCO_GL
NVIDIA_VISIBLE_DEVICES
NVIDIA_DRIVER_CAPABILITIES
```
```

次に mount の差分。

```
```bash id="ukp70u"
docker inspect good --format '{{json .Mounts}}' | python3 -m json.tool > good.mounts.json
docker inspect bad --format '{{json .Mounts}}' | python3 -m json.tool > bad.mounts.json

diff -u good.mounts.json bad.mounts.json | less
```
```

見るべき mount は主にこれです。

```
```text id="tco5p4"
/mnt/wslg
/tmp/.X11-unix
```
```

```
/usr/lib/wsl  
/dev/dri  
```
```

---

# 一番怪しいのはこれ

## 1. `/mnt/wslg` がマウントされていない

WSLg の Wayland socket や runtime file は `/mnt/wslg` 側にあります。

動くコンテナでは入っているのに、動かないコンテナにはない、というケースがかなりあります。

確認：

```
```bash id="40jw3t"  
docker exec good ls -l /mnt/wslg  
docker exec bad ls -l /mnt/wslg  
```
```

動かない側でこうなるなら怪しいです。

```
```text id="quf00d"  
No such file or directory  
```
```

または空っぽ。

---

## 2. `/tmp/.X11-unix` がマウントされていない

X11 経由で表示するアプリなら、ここが必要になることがあります。

確認：

```
```bash id="8a2dmh"  
docker exec good ls -l /tmp/.X11-unix  
docker exec bad ls -l /tmp/.X11-unix  
```
```

---

## 3. `DISPLAY` は同じでも、中身の socket が見えていない

例えば両方でこう出るとします。

```
```bash id="wsqfi0"  
DISPLAY=:0  
```
```

一見同じです。

でも `DISPLAY=:0` は「X server の 0 番を使う」という意味でしかありません。  
その実体である socket がコンテナ内から見えていなければ表示できません。

つまり、

```
```bash id="jk6l3v"
DISPLAY=:0
```
```

だけ渡しても不十分です。

必要なのは、少なくともこの組み合わせです。

```
```bash id="u1x6ps"
-e DISPLAY=$DISPLAY
-v /tmp/.X11-unix:/tmp/.X11-unix
```
```

WSLg/Wayland 側も使うなら：

```
```bash id="0nttce"
-e WAYLAND_DISPLAY=$WAYLAND_DISPLAY
-e XDG_RUNTIME_DIR=$XDG_RUNTIME_DIR
-v /mnt/wslg:/mnt/wslg
```
```

---

# まず試す docker run 例

動かないコンテナを一度この形で起動してみてください。

```
```bash id="lztvkj"
docker run -it --rm -e DISPLAY=$DISPLAY -e WAYLAND_DISPLAY=$WAYLAND_DISPLAY -e
XDG_RUNTIME_DIR=$XDG_RUNTIME_DIR -e PULSE_SERVER=$PULSE_SERVER -v /tmp/.X11-unix:/tmp/.X11-
unix -v /mnt/wslg:/mnt/wslg -v /usr/lib/wsl:/usr/lib/wsl -e LD_LIBRARY_PATH=/usr/lib/wsl/lib:
$LD_LIBRARY_PATH --device /dev/dri your_image:tag bash
```
```

コンテナ内で：

```
```bash id="k2nn14"
apt update
apt install -y x11-apps mesa-utils
```

```
xeyes
glxinfo -B
```
```

これで `xeyes` が出るか確認してください。

---

# 重要な切り分け



## A. `xeyes` も出ない

これはシミュレータ以前の問題です。

原因候補：

```
```text id="pi518a"
DISPLAY がない
/tmp/.X11-unix がない
/mnt/wslg がない
XDG_RUNTIME_DIR が間違っている
Wayland socket が見えていない
コンテナの実行ユーザー権限が違う
```
```

確認：

```
```bash id="z3kyj8"
echo $DISPLAY
echo $WAYLAND_DISPLAY
echo $XDG_RUNTIME_DIR
```

```
ls -l /tmp/.X11-unix
ls -l /mnt/wslg
ls -l $XDG_RUNTIME_DIR
```
```

---

## B. `xeyes` は出るが、シミュレータは出ない

この場合、GUI 転送自体は生きています。  
問題は **\*\*シミュレータの描画バックエンド\*\*** です。

よくあるのは：

```
```text id="s6q1sn"
OpenGL が使えない
Mesa / libGL が不足
Qt の xcb plugin が不足
pygame / SDL が Wayland を掴んで失敗
MuJoCo / GLFW が backend 選択に失敗
```
```

まず X11 に寄せて起動します。

```
```bash id="lw95c"
unset WAYLAND_DISPLAY
export SDL_VIDEODRIVER=x11
export QT_QPA_PLATFORM=xcb
export GDK_BACKEND=x11
```

```
python your_simulator.py
```

```

MuJoCo 系なら：

```
```bash id="c2af0v"
export MUJOCO_GL=glfw
python your_simulator.py
```
```

ダメなら software rendering：

```
```bash id="424n62"
export LIBGL_ALWAYS_SOFTWARE=1
python your_simulator.py
```
```

---

# Docker image の違いで起きる典型例

## 1. 動かないイメージに GUI ライブラリが入っていない

例えば Ubuntu slim / Python slim 系だと、GUI 周辺ライブラリがかなり削られています。

入れてみる候補：

```
```bash id="r560rf"
apt update
apt install -y x11-apps mesa-utils libgl1 libglx-mesa0 libegl1 libx11-6 libxext6 libxrender1 libxi6
libxrandr2 libxinerama1 libxcursor1 libxkbcommon-x11-0 libxcb-cursor0 libglb2.0-0
```
```

Qt 系なら追加で：

```
```bash id="r1lpc6"
apt install -y libxcb-xinerama0 libxcb-icccm4 libxcb-image0 libxcb-keysyms1 libxcb-render-util0 libxcb-
shape0 libxcb-randr0 libxcb-xfixes0
```
```

---

## 2. `--device /dev/dri` がない

OpenGL acceleration が必要な場合、これがないと失敗することがあります。

確認：

```
```bash id="9pbgor"
docker exec good ls -l /dev/dri
docker exec bad ls -l /dev/dri
```
```

動かない方に `/dev/dri` がなければ、起動時に追加。

```
```bash id="sfjlxj"
--device /dev/dri
```
```

---

## 3. `/usr/lib/wsl/lib` が見えていない

WSLg / WSL2 の OpenGL 周りでは、WSL 側のライブラリが効いている場合があります。

確認：

```
```bash id="2m8zer"
docker exec good ls -l /usr/lib/wsl/lib
docker exec bad  ls -l /usr/lib/wsl/lib
```
```

動かない方で存在しないなら、起動時に：

```
```bash id="s22m1p"
-v /usr/lib/wsl:/usr/lib/wsl
-e LD_LIBRARY_PATH=/usr/lib/wsl/lib:$LD_LIBRARY_PATH
```
```

---

# かなり実用的な比較コマンド

動くコンテナと動かないコンテナで、これを比較してください。

```
```bash id="dvfm0j"
for c in good bad; do
  echo "=====$c ENV===="
  docker exec $c sh -lc 'env | grep -E "DISPLAY|WAYLAND|XDG|PULSE|LD_LIBRARY|QT|SDL|GDK|MURJOCO|
NVIDIA" | sort'

  echo "=====$c SOCKETS===="
  docker exec $c sh -lc 'ls -ld /mnt/wslg/mnt/wslg/runtime-dir /tmp/.X11-unix 2>&1'

  echo "=====$c GPU/GL===="
  docker exec $c sh -lc 'ls -l /dev/dri 2>&1; ldconfig -p 2>/dev/null | grep -E "libGL|libEGL|libX11" | head -20'

  echo
done
```
```

差分が出たところがほぼ原因です。

---

# たぶん今回の本命

君の説明から見ると、優先順位はこうです。

| 優先度 | 原因                                                      | 理由                            |
|-----|---------------------------------------------------------|-------------------------------|
| 1   | `/mnt/wslg` or `/tmp/.X11-unix` の mount 差               | コンテナごとに設定差が出やすい               |
| 2   | `DISPLAY`, `WAYLAND_DISPLAY`, `XDG_RUNTIME_DIR` の env 差 | IP/ポート同じでも env がなければ表示不可      |
| 3   | image 内の GUI / OpenGL ライブラリ不足                           | image が違うならかなりあり得る            |
| 4   | `/dev/dri` や `/usr/lib/wsl` の有無                         | OpenGL シミュレータなら特に怪しい          |
| 5   | Wayland/X11 backend の違い                                 | 片方の image だけライブラリ構成が違っていると起きる |

---

#### # 最短の確認手順

動かないコンテナ内でこれを実行してください。

```
```bash id="9io2b8"
echo $DISPLAY
echo $WAYLAND_DISPLAY
echo $XDG_RUNTIME_DIR
```

```
ls -l /mnt/wslg
ls -l /mnt/wslg/runtime-dir
ls -l /tmp/.X11-unix
ls -l /dev/dri
```

```
xeyes
glxinfo -B
```
```

そして、動くコンテナでも同じことをやる。

**\*\*動く方にはあって、動かない方にはないもの\*\*が原因です。**

特に `ポート/IPが同じ`なのに片方だけ表示されないなら、ネットワークではなく **\*\*mount / env / library / device\*\*** の差を疑うべきです。